



School of Computing

GAME PROJECT REPORT

BRIDGES PUZZLE (HASHIWOKAKERO)

A Game with CPU Strategies Using Dynamic Programming, Divide & Conquer

TEAM - 4

TEAM MEMBERS :

DEVARA KISHORE REDDY - CB.SC.U4CSE24010

SANTHOSH.V - CB.SC.U4CSE24047

TULASI SADHWIKA - CB.SC.U4CSE24054

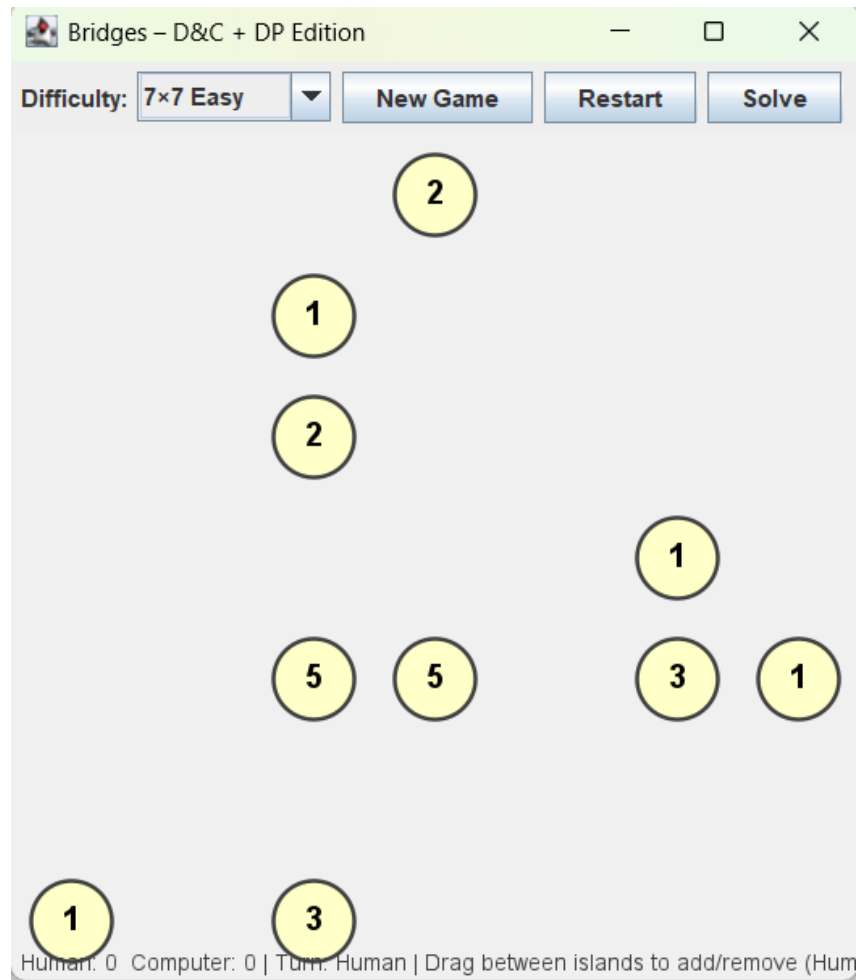
B.HASINI - CB.SC.U4CSE24068

CONTENTS

1	Introduction	3
1.1	Game Board	3
1.2	Project Objective	3
1.3	Game Overview	4
1.3.1	Game Rules	4
1.3.2	Game Components	4
2	Divide and Conquer Strategy	5
2.1	Why Sorting Is Needed In The Game?	5
2.2	How Merge Sort Is Applied In The Project	5
2.2.1	Divide	6
2.2.2	Conquer	6
2.2.3	Recursive Merge	6
2.3	Recurrence relation	7
2.4	Base case in DC	7
2.5	Why Divide And Conquer Is Important In This Game?	7
2.6	Time Complexity	8
2.7	Space Complexity	8
2.8	Why This Is Not Greedy	8
3	Dynamic Programming Strategy	9
3.1	Why Dynamic Programming is Needed?	9
3.2	Dp Idea In This Project:	10
3.2.1	Subproblem Definition:	10
3.2.2	Base Case	10
3.2.3	Recursive Case	11
3.2.4	Recurrence Relation	11
3.3	Time Complexity	12
3.4	Why This Is Dp	12
3.5	Correctness Proof	12
3.5.1	Base	12
3.5.2	Recursive	12
3.5.3	Termination	12
3.6	Space Complexity	13
3.7	Why This Is Not Greedy	13
4	Comparision of Strategies	14
5	Conclusion	15

1 INTRODUCTION

1.1 GAME BOARD



1.2 PROJECT OBJECTIVE:

The objective of this project is to design and implement a **Bridges (Hashiwokakero) puzzle game** integrated with algorithmic concepts from Design and Analysis of Algorithms (DAA).

The main goals of the project are:

- To model the Bridges puzzle as a graph problem.
- To implement an AI solver using **Dynamic Programming with memoization**.
- To apply **Divide and Conquer** strategy using Merge Sort.
- To analyze time and space complexity of the implemented algorithms.
- To demonstrate practical application of DAA concepts in game development.

The project aims to combine:

- Graph theory
- Recursion
- Memoization (DP)
- Divide and Conquer
- Algorithm complexity analysis

1.3 Game Overview:

What is Bridges (Hashiwokakero)?

Bridges is a logic-based puzzle played on a grid containing islands. Each island has a number indicating how many bridges must connect to it.

1.3.1 GAME RULES:

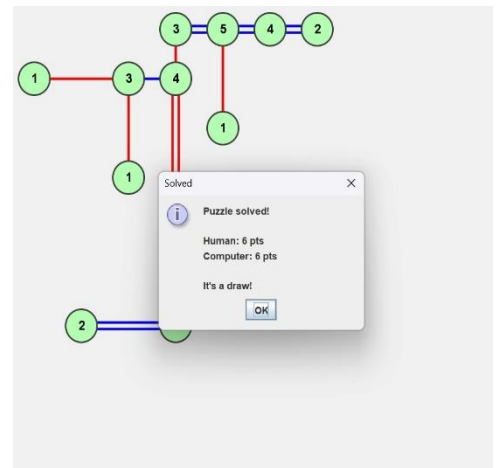
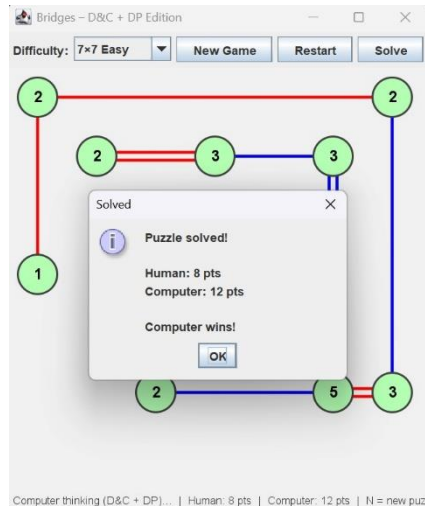
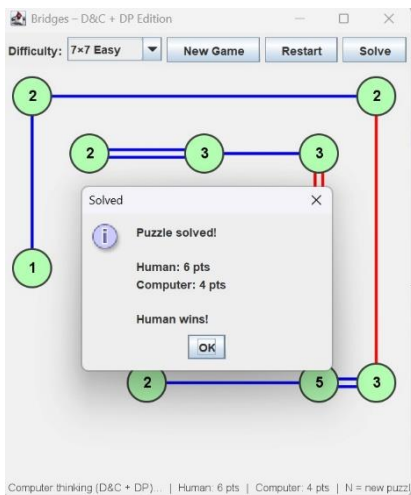
- Each island has a number indicating the required number of bridges
- Bridges can only be placed horizontally or vertically
- Bridges cannot cross each other
- A Maximum of two bridges can connect the same pair of islands.
- The puzzle is solved only when:
 - All islands are connected
 - Each island satisfies its required bridge count.

In our project:

- The Human player manually adds bridges.
- The AI solves the puzzle automatically using algorithmic techniques.

1.3.2 Game Components in the Project

- Islands → Represented as graph nodes.
- Bridges → Represented as graph edges.
- AI Solver → Uses Dynamic Programming with memoization.
- Sorting Module → Uses Merge Sort (Divide & Conquer).
- Validation Module → Uses DFS to check connectivity.



2 DIVIDE AND CONQUER STRATEGY:

In our Bridges (Hashiwokakero) project, Divide and Conquer is used mainly in:

1. Sorting islands
2. Sorting bridge candidates
3. Efficient organization of game elements using Merge Sort.

2.1 WHY SORTING IS NEEDED IN THE GAME?

In the Bridges puzzle:

- Islands must be checked in order (row-wise or column-wise).
- Bridge candidates must be processed in structured order.
- CPU needs predictable processing of nodes.

To achieve efficient sorting, we use:

Merge Sort (Divide and Conquer algorithm)

Divide and Conquer follows three steps:

1. **Divide** : Break the problem into smaller subproblems.
2. **Conquer**: Solve each subproblem recursively
3. **Combine**: Merge the solutions of subproblem

2.2 HOW MERGE SORT IS APPLIED IN THE PROJECT:

Suppose we have:

```
Island[] islands
```

We sort islands based on:

- X-coordinate
- Y-coordinate
- Priority (remaining bridges)

2.2.1 Step 1: Divide

Split array into two halves:

```
Left half → islands[0 to mid]
Right half → islands[mid+1 to end]
```

2.2.2 Step 2: Conquer

Recursively sort both halves:

```
mergeSort(left)
mergeSort(right)
```

Each half is again divided until:

```
Single element array
```

(Base case)

2.2.3 Step 3: Recursive merge

Merge two sorted halves:

```
merge(left, right)
```

Compare elements and arrange in sorted order.

2.3 Recurrence Relation

For Merge Sort:

$$T(n) = 2T(n/2) + O(n)$$

Where:

- $2T(n/2) \rightarrow$ sorting two halves
- $O(n) \rightarrow$ merging step

Using Master Theorem:

```
a = 2
b = 2
f(n) = n

n^(log_b a) = n^(log_2 2) = n
```

So:

$$T(n) = O(n \log n)$$

2.4 Base Case in Divide and Conquer

```
If array size <= 1
→ already sorted
→ return
```

This stops recursion.

2.5 WHY DIVIDE AND CONQUER IS IMPORTANT IN THIS GAME?

1. Improves Efficiency

Instead of using:

- Bubble Sort $\rightarrow O(n^2)$
- Selection Sort $\rightarrow O(n^2)$

We use:

2.6 TIME COMPLEXITY: $O(n \log n)$

Much faster for large boards.

2. Structured CPU Processing

Sorting islands ensures:

- Consistent bridge generation
- Deterministic AI behavior
- Organized traversal

3. Reduces Complexity in CPU Decisions

Sorted islands help:

- Quickly find nearest neighbors
- Avoid unnecessary comparisons
- Improve bridge validation

2.7 SPACE COMPLEXITY OF DIVIDE AND CONQUER

Merge Sort requires:

$O(n)$ auxiliary space

Because temporary arrays are used during merging.

2.8 WHY THIS IS NOT GREEDY ?

What Would Greedy Sorting Look Like?

Greedy sorting examples:

- Selection Sort → pick smallest element each iteration
- Insertion Sort → insert element in best local position

These algorithms make **local optimal choices** step by step.

What Merge Sort Does:

1. Divides array into two halves
2. Recursively sorts both halves
3. Merges sorted halves

It does NOT:

- Pick the globally smallest element immediately
- Decide based on local optimality

Why It Is NOT Greedy

Because:

- It does not choose best element from entire array first
- It does not build final solution step by step using local best
- It solves subproblems independently before merging

Greedy would say:

“Pick smallest element, fix it, repeat.”

Merge Sort does not do that.

3 DYNAMIC PROGRAMMING STRATEGY:

3.1 Why Dynamic Programming is Needed?

The Bridges puzzle requires:

- Trying different bridge combinations.
- Checking if a partial configuration can lead to a valid full solution.
- Avoiding invalid configurations.
- Avoiding recomputation of already explored states.

If we use simple recursion (backtracking):

Time Complexity → Exponential $O(2^B)$

Many states are recomputed multiple times.

To optimize this, we use:

Dynamic Programming with Memoization

3.2 Dp Idea In This Project:

We break the puzzle into smaller subproblems:

"Can the puzzle be solved from the current partial bridge configuration?"

If we solve this once, we store the result and reuse it.

3.2.1 Subproblem Definition:

Let:

DP(state) = true, if puzzle is solvable from current state

false, otherwise

Where state contains:

- Remaining bridge requirement of each island
- Current bridge placements
- Connectivity structure of graph

3.2.2 Base Case:

The recursion stops when:

Case 1: Valid Solution Found

If remaining_degree[i] = 0 for all islands i

AND graph is fully connected

→ return TRUE

Case 2: Invalid Configuration

If:

- Any island degree becomes negative
- More than 2 bridges placed between same pair
- Bridges cross each other
- Graph becomes disconnected permanently

→ return FALSE

3.2.3 Recursive Case:

From the current state:

1. Select an island with remaining bridges.
2. Try placing:
 - 1 bridge
 - 2 bridges (if allowed)

For each valid placement:

Create new_state

Call DP(new_state)

Then:

DP(current_state) =

OR of all DP(new_state)

If any recursive call returns TRUE → solution exists.

3.2.4 Recurrence Relation:

Let B = number of possible bridge decisions.

Without DP:

$$T(B) = 2T(B - 1) + O(1)$$

Solving:

$$T(B) = O(2^B)$$

Exponential growth.

With Memoization:

Each unique state is computed only once.

Let S = number of valid unique states.

Then:

$$T(S) = O(S)$$

But each state also checks connectivity using DFS:

$DFS = O(V + E)$

$S \leq 2^B$ (worst case)

So total:

3.3 TIME COMPLEXITY:

$O(S \times (V + E))$

3.4 Why This is Dynamic Programming?

Because:

- Problem has Overlapping Subproblems
Same bridge configuration can be reached in different ways.
- Problem has Optimal Substructure
If a partial configuration leads to a solution, the full solution depends on it.
- We use Memoization
Store results of computed states to avoid recomputation.

3.5 CORRECTNESS PROOF:

3.5.1 Base:

If all islands satisfy required degree and graph is connected $\rightarrow$ correct solution.

3.5.2 Maintenance:

Each recursive step:

- Maintains degree constraints.
- Ensures no crossing bridges.
- Ensures max two bridges between islands.

Thus invariant is preserved.

3.5.3 Termination:

- Total remaining degree decreases in each step.
- Finite number of states.
- Memoization avoids infinite loops.

Hence algorithm terminates.

3.6 SPACE COMPLEXITY:

- 1.DP memo table: $O(S)$
- 2.Recursion stack: $O(N)$
- 3.Graph storage: $O(V + E)$
- 4.Total: $O(S + V + E)$

3.7 WHY THIS IS NOT GREEDY?

What is a Greedy Algorithm?

A Greedy algorithm:

- Makes the **locally optimal choice** at each step.
- Does NOT reconsider decisions.
- Does NOT explore multiple possibilities.
- Does NOT store states.

Example:

“Always connect the island with maximum possible bridges first.”

That is greedy.

What Our DP Solver Does

Our Bridges CPU:

- Defines subproblem: $DP(\text{state})$
- Tries different valid bridge placements
- Recursively checks future feasibility
- Uses memoization to store results
- May reject a locally valid move if it leads to dead end

This means:

- It does NOT commit to first valid choice.
- It evaluates future consequences.
- It explores state space systematically.

Why It Is NOT Greedy?

Greedy makes:

Decision based only on current information.

Our DP:

Makes decision based on entire future solvability.

Greedy:

Choose best local option → move forward → never look back

DP:

Try option → check recursively → if fails, try another

Store result for reuse

4 COMPARISON OF STRATEGIES:

DP vs Divide & Conquer in Bridges Game

Aspect	Dynamic Programming	Divide & Conquer
Look-ahead	Yes – checks whether a bridge configuration can lead to a complete valid solution	None – only organizes islands/bridges for structured processing
Main Idea	Define subproblem DP(state) and use memoization to avoid recomputation	Divide island/bridge list into halves, sort recursively using merge sort, merge results
Decision Process	Recursively evaluates all valid next configurations and stores results	Divides problem into smaller sublists and combines sorted results
Time Complexity	$O(S \cdot (V + E)) \approx O(2^B)$ worst case	$O(n \log n)$
Space Complexity	$O(S + V + E)$ (memo + recursion stack)	$O(n)$ auxiliary space
Greedy?	No (checks global feasibility + memoization)	No (divide–conquer–combine structure)

5 CONCLUSION:

In this project, we implemented the Bridges (Hashiwokakero) puzzle using core DAA concepts. The AI solver applies Dynamic Programming with memoization to avoid recomputation of identical board states and ensure efficient solution evaluation. Additionally, Divide and Conquer is used through Merge Sort to organize islands and bridge candidates efficiently.

The DP approach reduces redundant calculations, while the D&C strategy ensures structured and efficient processing. Overall, the project demonstrates the practical application of classical algorithmic techniques in a game AI system.

THANK YOU